

Specification-Driven Development with OpenSpec

"Agree before you build"

Andrii Sukhoi

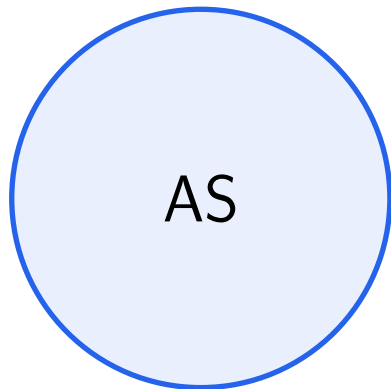
Lubelski Klub IT

2026

Andrii Sukhoi

- Senior Software Engineer
- CEO of Dusha Fund
- Tennis Instructor
- Running 5 AI agents inside myself

github.com/sukhoi94



> 90%

of developers now use AI coding assistants

— GitHub Developer Survey, 2025

But There's a Problem

AI generates code
without formal requirements.

It works from whatever you typed in chat.
No structure. No memory. No contract.

Chat history is not a specification.

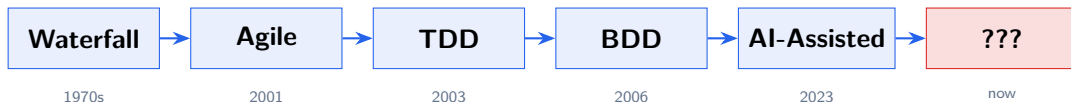
?

Requirements get lost, mutate,
contradict each other across messages.

*"Vague prompts lead to
unpredictable results."*

— OpenSpec Documentation

The Evolution of Software Development



Each era solved the previous era's problem.

But AI created a new gap.

Waterfall's Problem

Waterfall failed because...

Requirements frozen at day one.

Customer sees product after **years**.

Feedback loop: **too late**.

Agile solved it:

Short iterations (sprints).

Working software every 2 weeks.

Requirements can change.

"Responding to change over following a plan" — Agile Manifesto, 2001

Agile's Problem

Agile failed because...

Speed without safety net.

Regressions ship constantly.

"Move fast, break things."

Kent Beck popularized TDD in *Extreme Programming*, 1999.

TDD solved it:

Write tests first.

Red → Green → Refactor.

Safety net for fast iterations.

TDD's Problem

TDD failed because...

Tests written in code.

Business can't read them.

Gap between **intent** and **test**.

BDD solved it:

Human-readable scenarios.

GIVEN / WHEN / THEN

Business + Dev speak same language.

Dan North introduced BDD in 2006. Cucumber, SpecFlow born from this.

BDD's Problem

BDD failed because...

Writing scenarios is slow.

Maintenance overhead grows.

Adoption drops in fast teams.

GitHub Copilot (2021), ChatGPT (2022), Claude, Cursor...

AI-Assisted solved it:

AI writes boilerplate fast.

Code generation at scale.

Developer productivity ↑↑

AI-Assisted's Problem

AI-Assisted fails because...

AI works from chat history.

No formal requirements.

Fast, but **in the wrong direction**.

That's where OpenSpec comes in.

SDD solves it:

Specs before code.

Human reviews artifacts.

AI implements **what was agreed**.

Waterfall Had Specs

Waterfall gave us formal specifications.

But they were:

- Written once, **never updated**
- Disconnected from code
- Dead on arrival

*"Working software over
comprehensive documentation"*

— Agile Manifesto, 2001

User stories replaced specs.
Conversations replaced documents.
Speed replaced formality.

TDD: Tests as Specs

Tests define behavior, but not intent.

*"Testing shows the presence,
not the absence of bugs."*

— Edsger W. Dijkstra, NATO Conference, 1969

Tests verify **what** the code does.

They don't explain **why** it should do it.

Productivity



Clarity



We can build faster than ever.
But we lost track of **what** we're building.

When Specs Are Missing: Therac-25

1985–1987. A radiation therapy machine.

- No formal specifications for concurrency
- Race condition in software → massive overdose
- **6 patients died**

"The most serious computer-related accidents ever investigated." — Leveson & Turner, IEEE Computer, 1993

The Cost of a Missing Spec

Phase	Cost to fix
Design	1x
Development	10x
Production	100x

— IBM Systems Sciences Institute

A missing spec is the most expensive bug you'll never find.

Specification-Driven Development

Write formal specs **before** code.

But iterate on them — this is **NOT waterfall**.

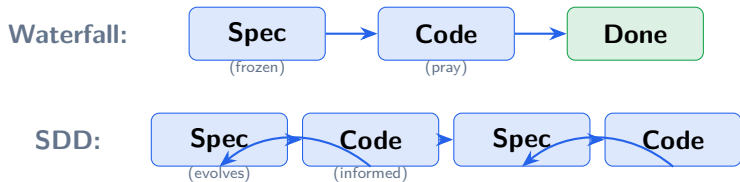
Specs are living documents
that evolve with the project.

Formal Specs Are Not New

- **NASA Apollo** — every line of code had a formal requirement.
Apollo 11 landed with 4KB RAM. Zero unspecified behavior.
- **TLA+** (Leslie Lamport, Turing Award 2013) — formal spec language
Used by Amazon Web Services to verify DynamoDB and S3.
- **OpenSpec** — lightweight SDD for the AI era

Specs are not a new idea. **AI made them urgent again.**

SDD is NOT Waterfall



Specs and code evolve together through **feedback loops**.

SDD vs TDD vs BDD

Approach	Spec Source	Written When	AI-Native?
No specs	Chat history	Never	Poor
TDD	Test cases	Before code	Partial
BDD	User stories	Before tests	Partial
SDD	Formal specs	Before everything	Yes

SDD is the first approach built **for** the AI era.

Why Write Things Down?

*"If you're thinking without writing,
you only think you're thinking."*

— Leslie Lamport,
Specifying Systems, 2002

What is OpenSpec?

OpenSpec

An AI-native framework
for specification-driven development.

MIT licensed · TypeScript · 36+ AI tools supported

The Numbers

71K+

GitHub stars

265K+

developers/month

36+

supported AI tools

*"One new spec created
every two seconds."*

— openspec.dev

Fluid not rigid

Iterative not waterfall

Easy not complex

Brownfield not just greenfield

Scalable solo dev to enterprise

Let's see it in action.

Install in 30 Seconds

```
npm install -g @fission-ai/openspec  
cd your-project  
openspec init  
openspec update
```

Done. Your AI tools are now spec-aware.

The Mental Model



Every change is **planned before built**,
reviewed before implemented,
archived after done.

Specs = Requirements + Scenarios

Plain Markdown files defining what the system SHALL do.

```
## Requirements
- SHALL support light and dark themes
- SHALL persist user preference

## Scenarios
- WHEN user toggles theme
  THEN UI updates immediately
- WHEN user reopens app
  THEN last theme is restored
```

A Change = A Self-Contained Proposal

```
openspec/changes/add-dark-mode/  
|- proposal.md  
|- design.md  
|- tasks.md  
`- specs/
```

Everything about one change lives in one folder.

proposal.md — The Why and What

- What problem are we solving?
- What is **in scope**?
- What is explicitly **OUT of scope**?

The proposal is the contract between human and AI.

design.md — The How

- Technical approach
- Architecture decisions
- Trade-offs considered

Written by AI, **reviewed by human.**

tasks.md — The Checklist

- [] Add CSS custom properties for colors
- [] Create ThemeToggle component
- [] Add localStorage persistence
- [] Update components to use theme variables
- [] Add tests for theme switching

Discrete, implementable units of work.

**"AI creates artifacts.
Human reviews.
AI implements."**

How to Read a Spec

Read in this order:

- 1 Purpose** — understand *why* before *what*.
If you don't understand the why, the requirements won't make sense.
- 2 Out of Scope** — know the boundaries first.
Scope defines what the spec is NOT responsible for.
- 3 Requirements** — each SHALL is a contract.
Read as: "The system is obligated to..."
- 4 Scenarios** — these show how requirements behave in practice.
WHEN/THEN are the edge cases hiding in plain sight.

Green Flags: This Spec is Good

- **Testable** — "SHALL respond within 200ms"
Not: "SHALL be fast"
- **One responsibility** — fits on one screen, one capability
- **Scenarios cover edge cases** — not just the happy path
- **No HOW, only WHAT** — requirements don't dictate implementation
- **Explicit out of scope** — you know what it deliberately ignores

Red Flags: This Spec is Bad

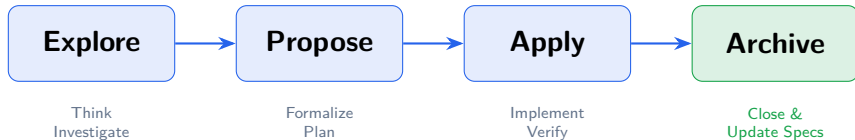
- **Vague language** — "user-friendly", "fast", "secure", "intuitive"
These cannot be tested. They mean nothing.
- **Implementation leaking in** — "SHALL use Redis for caching"
That's design, not a requirement.
- **Missing scenarios** — requirements without examples
If there are no scenarios, no one agreed on the details.
- **Weasel words** — "should", "might", "usually", "as needed"
SHALL is mandatory. "should" means maybe.

The Smell Test

Ask these questions when reviewing a spec:

Question	Yes	No
Can I write a test for this requirement?	Good	Rewrite
Does it say WHY, not HOW?	Good	Rewrite
Does every requirement have a scenario?	Good	Add one
Is "out of scope" section present?	Good	Add it
Fits on one screen?	Good	Split it

The Four Phases



A full cycle from idea to done.

/opsx:explore

- Investigate the codebase
- Ask questions, think freely
- No commitments, no files written
- Pure discovery

"Thinking time, not task time."

Propose

```
/opsx:propose add-dark-mode
```

- Creates change folder with all artifacts
- AI drafts proposal, design, specs, tasks
- Human reviews and approves each artifact

Nothing is built until YOU say so.

/opsx:apply

- Implements tasks one by one
- Each task verified before moving to next
- AI follows the design, not its imagination

The spec is the guardrail.

/opsx:archive

- Change is completed and archived
- Main specs updated with new requirements
- History preserved for future reference

The loop closes. Specs are the living record.

Example: "Add Dark Mode"

We want to add **dark mode** to our web app.

Without specs:

"Hey AI, add dark mode" → unpredictable result

With OpenSpec:

structured, reviewed, implemented step by step

The Proposal

proposal.md

```
## Summary
Add dark mode with user preference persistence.

## In Scope
- Theme toggle component
- CSS custom properties for theming
- localStorage persistence

## Out of Scope
- System preference detection (future)
- Per-page theme overrides
```

Clear boundaries. AI knows what **NOT** to do.

The Design

design.md

```
## Approach
Use CSS custom properties on :root.
Toggle class on <html> element.

## Decision: CSS Variables vs CSS-in-JS
CSS variables chosen for:
- Zero runtime cost
- Works with existing stylesheets
- No library dependency
```

Decisions documented, not buried in comments.

The Tasks

`tasks.md`

- [] Define color tokens in :root and [data-theme="dark"]
- [] Create ThemeToggle component
- [] Wire toggle to localStorage
- [] Migrate hardcoded colors
- [] Add unit tests for ThemeToggle

Each task is independently implementable and verifiable.

After Archive

Before:

```
specs/  
(no theming spec)
```

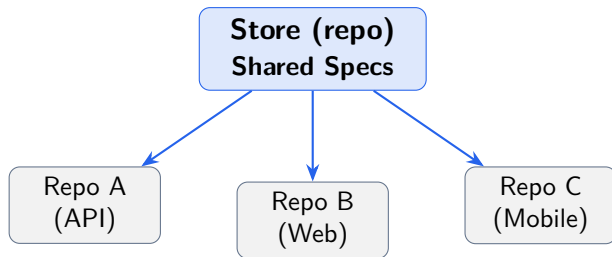
After:

```
specs/theming/spec.md  
- SHALL support light  
  and dark themes  
- SHALL persist pref  
  in localStorage
```

The spec becomes the **permanent record**.

Next time anyone touches theming — the spec is there.

Stores: Cross-Repo Planning



Platform teams publish specs.
Product teams consume them.

36+ Supported Tools

Works with your **existing** setup.

Claude Code ▪ GitHub Copilot ▪ Cursor ▪ Amazon Q

Codeium ▪ Windsurf ▪ Aider ▪ Continue ▪ Zed

JetBrains AI ▪ VS Code ▪ and more...

No vendor lock-in. MIT licensed.

You don't need to start from scratch.

```
cd your-existing-project  
openspec init
```

Two commands. Your existing code stays untouched.

Specs describe what you build **next**, not what already exists.

openspec update

- Refreshes AI instruction files in your project
- Run after updating OpenSpec or changing config
- Ensures every AI tool reads the latest specs and rules

"Your AI assistants are only as good as their instructions."

vs GitHub Spec Kit

	OpenSpec	Spec Kit
Weight	Lightweight	Heavyweight
Workflow	Iterative	Rigid phase gates
Setup	<code>npm i -g openspec</code>	Python environment
Flexibility	Any AI tool	GitHub ecosystem

"Thorough but heavyweight." — OpenSpec comparison

vs AWS Kiro

	OpenSpec	Kiro
IDE	Any (36+ tools)	Kiro IDE only
Models	Any LLM	Claude only
Approach	Spec-first	IDE-integrated
Lock-in	None (MIT)	AWS ecosystem

"Powerful but locked into their IDE." — OpenSpec comparison

vs No Framework

	OpenSpec	No Framework
Requirements	Versioned Markdown	Chat history
Persistence	Git-tracked specs	Gone when chat closes
Review	Human approves	Hope for the best
Consistency	Enforced by specs	Random

"Hope is not a strategy."

Not everyone is convinced.

Let's be honest about the objections.

Controversy #1

"This is just waterfall in disguise."

The critique

Agile killed BDUF in 2001.
Writing specs first = Big Design Up Front.
We're going backwards.

The reality

OpenSpec specs *evolve*.
Waterfall specs were frozen.
Iterative $\neq$ specless.

Controversy #2

"AI doesn't actually read specs."

The critique

LLMs are probabilistic.
Context windows overflow.
AI "agrees" then ignores constraints.

The reality

OpenSpec is designed around this.
Structured context > chat soup.
Human review catches drift.

"Garbage in, garbage out."

The critique

A wrong spec perfectly implemented
is still wrong.
Who validates the specs?

The reality

This is actually a **feature**.
Bad assumptions become
explicit and traceable.

The Honest Answer

OpenSpec is not a silver bullet.

- It adds overhead — especially for tiny projects
- AI models still hallucinate, specs or not
- Bad specs produce bad software, faster

But: the alternative is unstructured prompting at scale. That's worse.

Why This Matters

AI without specs is chaos on a longer horizon.

One feature? Fine.

Ten features? Cracks appear.

A full product? **Unmanageable without specs.**

Specs $\neq$ Waterfall

Specification-Driven $\neq$ Waterfall

- Specs are lightweight Markdown
- They evolve iteratively
- They are AI-native — written and consumed by AI
- Human stays in control

"Agree before you build."

Questions?

GitHub: github.com/Fission-AI/openspec

Website: openspec.dev

Install: `npm install -g @fission-ai/openspec`

Me: github.com/sukhoy94